

Classes, instances & attributes

The core mental model: one class describes a type of object; each created instance has its own state.

Four terms to understand

TERM	MEANING
Class	Defines a type of object; written using <code>class</code> . Example: Python has a built-in <code>list</code> class.
Instance / object	One particular object created by calling the class name with <code>(...)</code> .
State / attribute	Data kept on an instance. Access it using <code>object.attribute</code> .
Method	A function defined inside the class; it can read or change instance state.

Create a class and two objects

CLASS DEFINITION AND CONSTRUCTION

```
class SimplestPossibleClass:
    pass

first = SimplestPossibleClass()
second = SimplestPossibleClass()
```

Remember: `pass` is a placeholder meaning “do nothing”; the colon introduces an indented class body. Defining the class is not the same as creating an instance.

Attributes belong to individual instances

ASSIGN AND READ OBJECT STATE

```
first.a = 3
second.b = 2
print(first.a)      # 3
print(second.b)     # 2
# print(first.b)    # AttributeError
```

Core idea. `first` and `second` are different objects. Adding `b` to the second instance does **not** add `b` to the first. Reading an attribute before it exists raises `AttributeError`.

Methods and the role of self

Methods are functions inside a class. For the instance methods in this notebook, Python supplies the receiving object automatically.

Define a method; call it through an instance

THE NOTEBOOK'S METHOD EXAMPLE (CONDENSED)

```
class ClassWithOneMethod:
    def mymethod(self, a, b):
        print(f"State: {self.example_state}")
        print(a, b)

obj1 = ClassWithOneMethod()
obj1.example_state = "red"
obj1.mymethod(1, 2) # prints "State: red" and "1 2"
```

How arguments are matched

For `obj1.mymethod(1, 2)`, the definition has **three parameters** but the call needs only **two explicit arguments**:

PARAMETER	RECEIVES
<code>self</code>	<code>obj1</code> - supplied automatically by Python
<code>a</code>	1 - first explicit argument
<code>b</code>	2 - second explicit argument

What `self` means. Inside the method, `self.example_state` reads the state of whichever instance was used to call it. `self` is a conventional parameter name, **not** a reserved keyword.

Common errors to recognise

ATTEMPT	WHY IT FAILS
<code>mymethod(obj1, 1, 2)</code>	<code>NameError</code> here: it was not defined as a standalone function.
<code>obj1.mymethod(obj1, 1, 2)</code>	Too many explicit arguments: <code>obj1</code> is already supplied as <code>self</code> .
<code>obj3.mymethod(1, 2)</code>	<code>AttributeError</code> if <code>obj3.example_state</code> has never been assigned.

Syntax reminder: a method is indented inside the class; its body is indented again. An f-string such as `f"State: {self.example_state}"` inserts the attribute's current value.

Read, return or modify?

The important distinction: a method may compute and return a result, or update data stored on the instance.

The Adder example

THREE METHODS, DIFFERENT BEHAVIOUR

```
class Adder:
    def add_a(self, b):
        return self.a + b

    def initialize_a(self, a):
        self.a = a

    def increment_a(self):
        self.a = self.a + 1
```

METHOD	ACTION	CHANGES `SELF.A`?
<code>initialize_a(a)</code>	Stores the given value.	Yes
<code>add_a(b)</code>	Returns <code>self.a + b</code> .	No
<code>increment_a()</code>	Adds 1 to stored value.	Yes

Do not confuse these. In `self.a = a`, the left side is a persistent **instance attribute**; the right side is this call's local parameter. In `self.a = self.a + 1`, the result is stored back on the object.

Follow two independent instances

READ THE COMMENTS AS EXPECTED RESULTS

```
adder1, adder2 = Adder(), Adder()
adder1.initialize_a(4) # adder1.a = 4
adder2.a = 6           # adder2.a = 6 (direct assignment)
print(adder1.add_a(5)) # 9; no change to adder1.a
adder1.increment_a()   # adder1.a is now 5
print(adder1.add_a(5)) # 10
print(adder2.add_a(5)) # 11; adder2.a is still 6
```

Return vs mutation. `add_a` returns a number but leaves state unchanged. `initialize_a` and `increment_a` change state and have no return, so their return value is `None` - that does **not** mean nothing happened.

Automatic setup with `__init__`

Use the special method `__init__` to initialise a new object as it is created, instead of calling a setup method separately.

Manual versus automatic initialisation

MANUAL / ADDER

```
adder = Adder()
adder.initialize_a(2)
# Two separate calls
```

AUTOMATIC / ADDERINIT

```
adder = AdderInit(2)
# __init__ runs for you
# Stored a starts at 2
```

Define the initializer

SAME BEHAVIOUR, LESS SETUP WHEN CONSTRUCTING

```
class AdderInit:
    def __init__(self, a):
        print("Inside __init__!!!")
        self.a = a

    def add_a(self, b):
        return self.a + b

    def increment_a(self):
        self.a = self.a + 1
```

`AdderInit(3)` creates a **new instance**, calls `__init__` automatically with `a = 3`, and stores 3 in that instance's attribute `a`.

CHECK THE RESULT

```
adder = AdderInit(3)    # prints: Inside __init__!!!
print(adder.a)          # 3
adder.add_a(6)          # returns 9; adder.a remains 3
```

Remember the spelling. `__init__` has **two underscores on each side**. Python recognises that exact name and calls it during construction. Unlike a method named `initialize_a`, you do not invoke it separately in the examples here.

Test yourself

Two compact exercises from the notebook, followed by their outputs and the rules worth revising.

1 / Predict the output

THE TWO-INSTANCES QUESTION

```
left, right = AdderInit(2), AdderInit(10)
result = left.add_a(5)
left.increment_a()
print(result)
print(left.a, right.a)
print(right.add_a(left.a))
print(left.a, right.a)
```

Answer (in order): the two `__init__` messages, then 7, 3 10, 13, 3 10. The saved number `result` stays 7; only `left.a` is incremented.

2 / Write a running-total Counter

Requirements: `__init__(start)` stores value; `add(amount)` **changes** it without returning a value; `read()` returns the current value without modifying it.

SOLUTION - TRY WRITING IT BEFORE READING

```
class Counter:
    def __init__(self, start):
        self.value = start

    def add(self, amount):
        self.value = self.value + amount

    def read(self):
        return self.value
```

STARTING OBJECTS / OPERATION	WHAT HAPPENS
<code>Counter(2)</code> and <code>Counter(10)</code>	Their values start at 2 and 10.
<code>counter1.add(3); read both</code>	5 10
<code>print(counter1.add(4)); read both</code>	None, then 9 10
<code>counter2.add(-2); read both</code>	9 8

Five-point revision check. (1) Class vs instance; (2) attributes store separate state; (3) `self` receives the caller; (4) **returning** a value is different from **mutating** an attribute; (5) `__init__` initialises each new instance automatically.